EXPERIMENT 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.

Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
   Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

6. Chain Replacement

When /replace_chain is called:

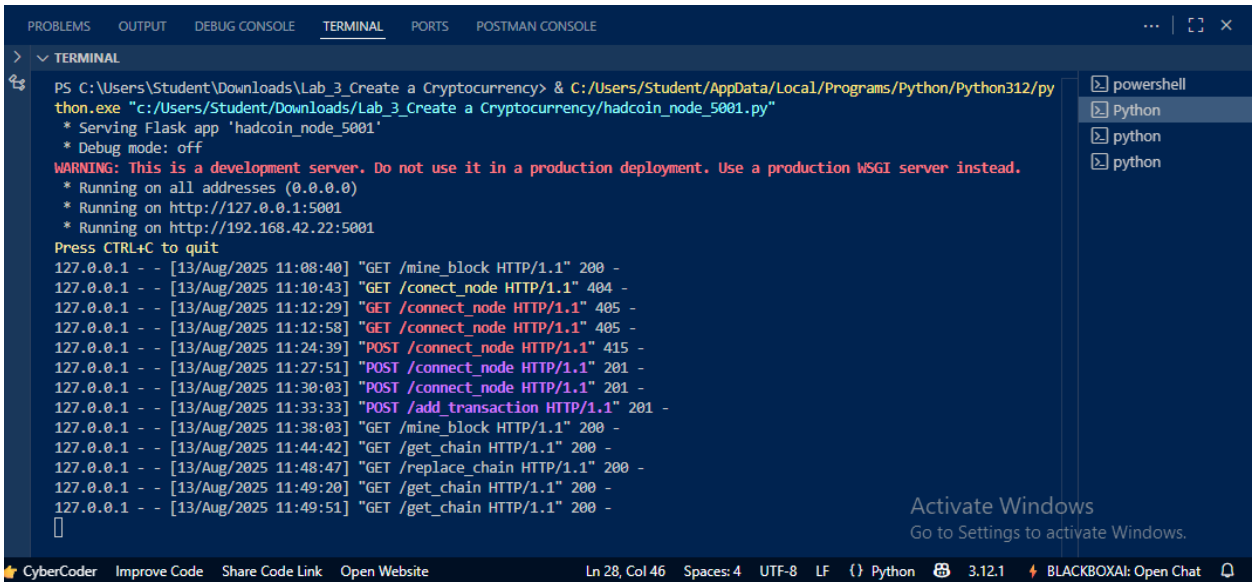
1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

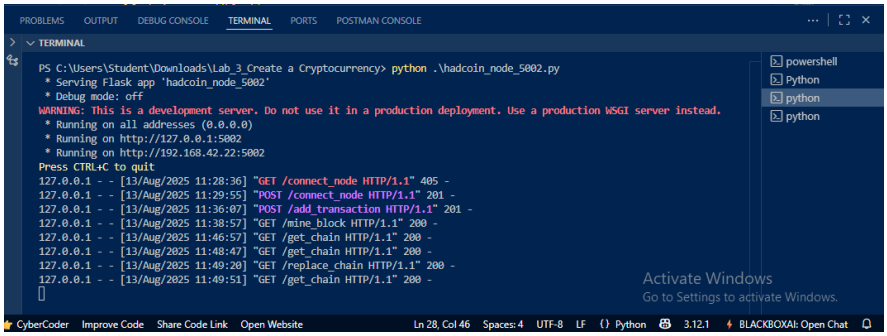
Tools & Libraries Used

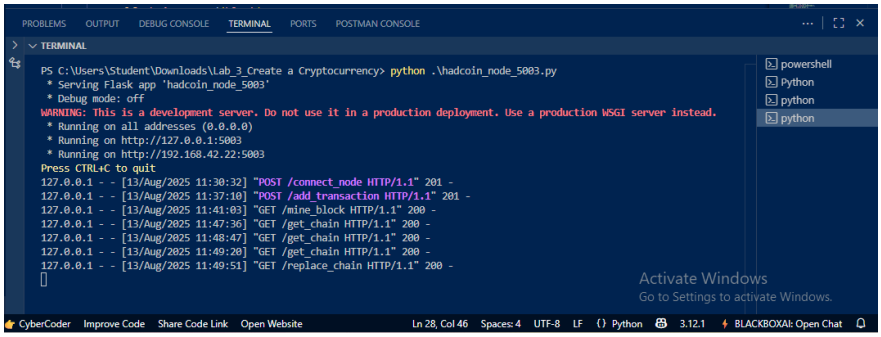
- **Python 3.x**
- **Flask** – Web framework for API endpoints
  pip install Flask
- **Requests** – For HTTP communication between nodes
  pip install requests==2.18.4
- **Postman** – For testing API requests
- Python Standard Libraries:
  - datetime
  - jsonify
  - hashlib
  - uuid4
  - urlparse
  - request

Procedure and Output:

1. Download hadcoin_node_5001.py, hadcoin_node_5002.py, hadcoin_node_5003.py.
2. Install required packages.
3. Run each node in separate terminals.



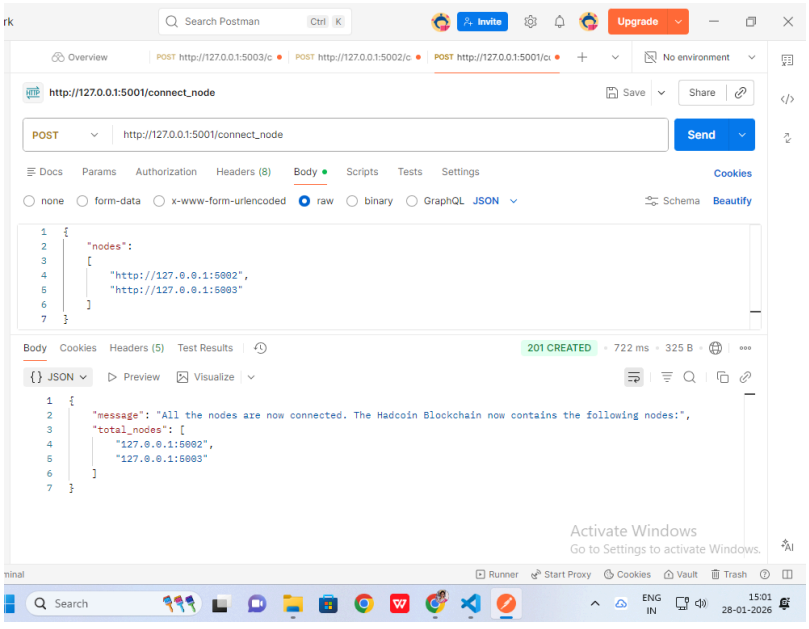


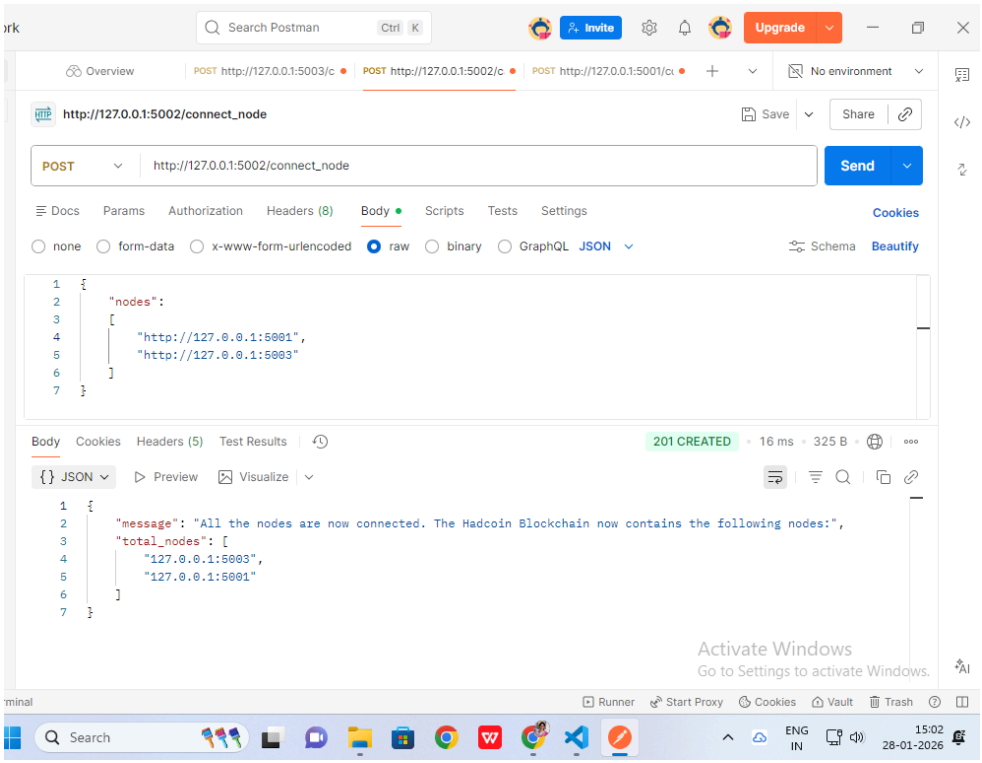


4. Connect nodes using Postman – POST /connect_node.

```
{
  "nodes":
  [
    "http://127.0.0.1:5002",
    "http://127.0.0.1:5002"
  ]
}
```

5.





6. Add transactions – POST
7. Add 3 transactions in 5001
8. Condition 2 — Each node must have **different chain lengths**

Example:

If you mined 1 block on 5001, but did not mine on 5002 or 5003 →

- **5001 chain length = 2**
- **5002 chain length = 1**
- **5003 chain length = 1**

STEP 2 — Perform all transactions ONLY from one node (e.g., 5001)

POST → http://127.0.0.1:5001/add_transaction

```
{
 "sender": "a",
 "receiver": "b",
 "amount": 5
}
```

Then 5001 → GET /mine_block

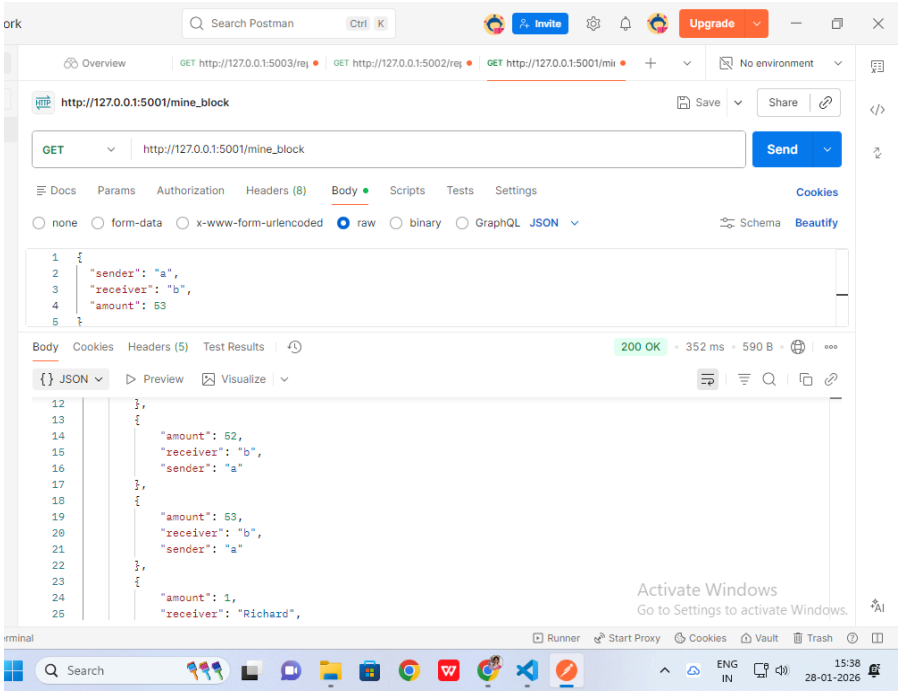
STEP 3 — Now go to 5002 and run:

GET → /replace_chain

STEP 4 — Repeat on 5003

GET → /replace_chain

Same result.

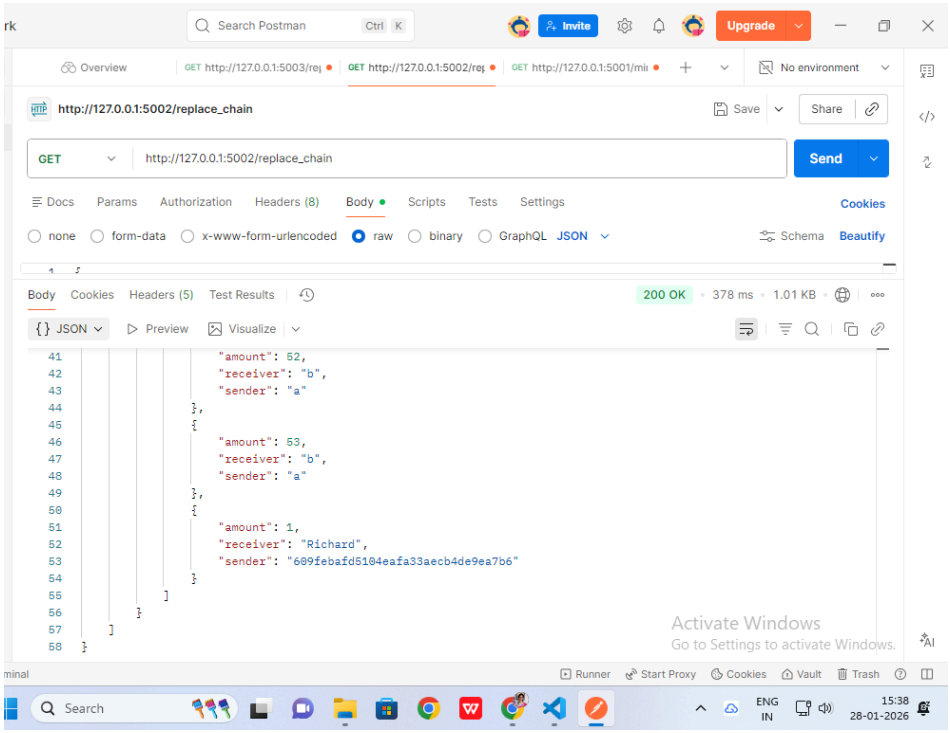


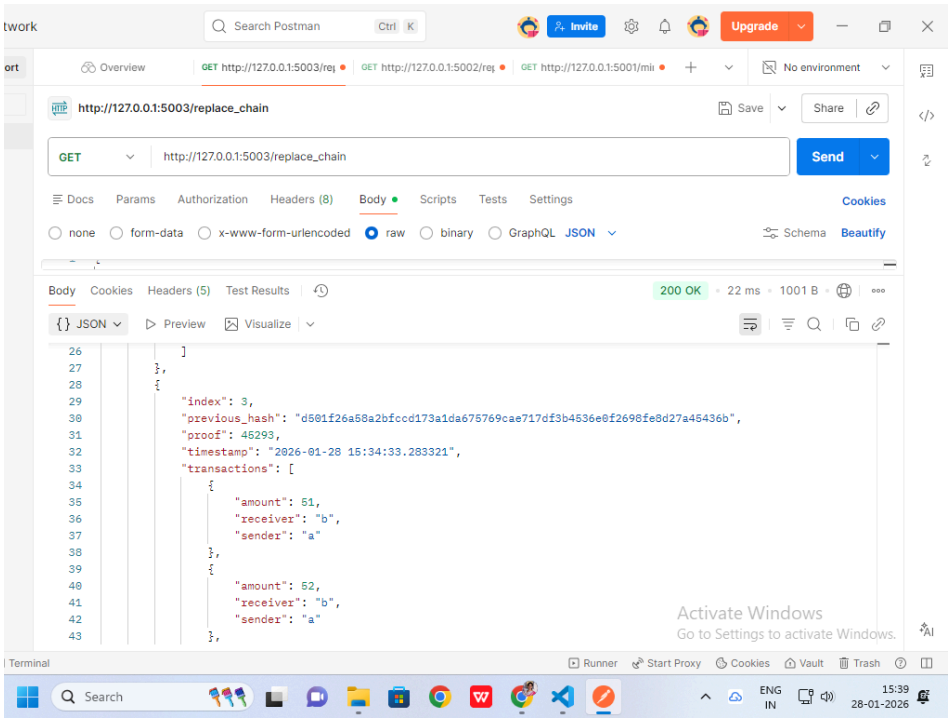

```
{
 "sender": "a",
 "receiver": "b",
 "amount": 53
}
```

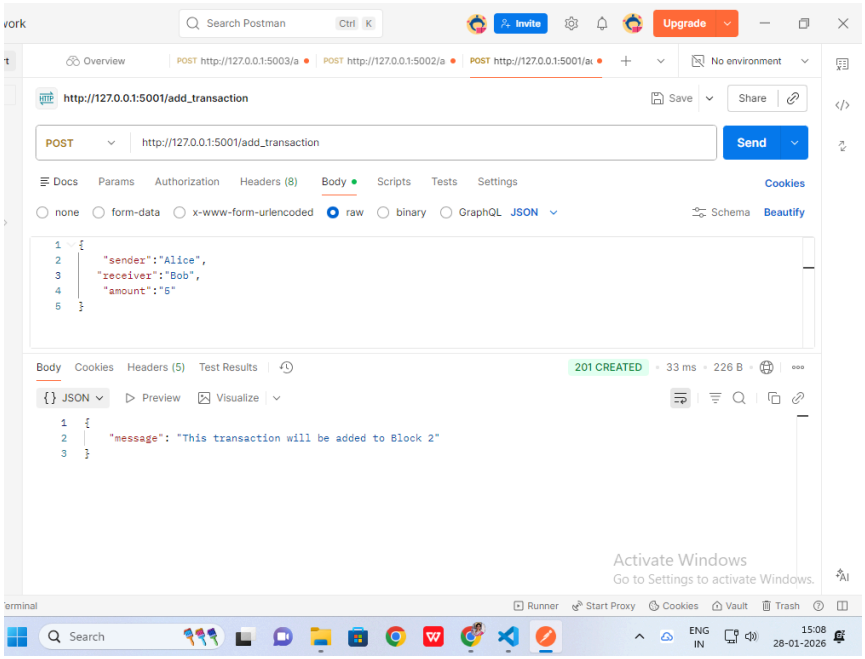

Output


```
{
  "index": 3,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "d501f26a58a2bfccd173a1da675769cae717df3b4536e0f2698fe8d27a45436b",
  "proof": 45293,
  "timestamp": "2026-01-28 15:34:33.283321",
  "transactions": [
    {
      "amount": 51,
      "receiver": "b",
      "sender": "a"
    },
```

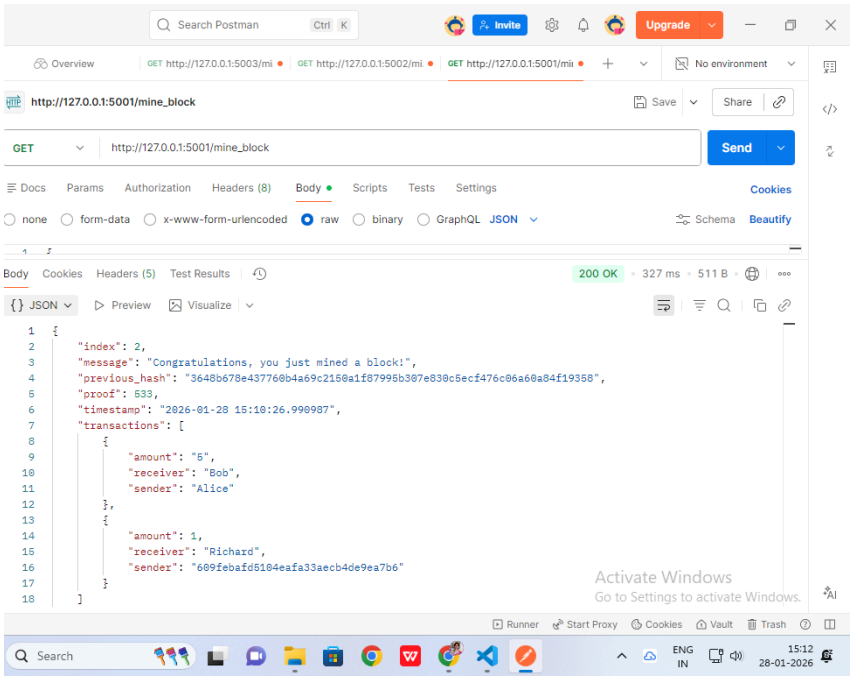
```
        {
            "amount": 52,
            "receiver": "b",
            "sender": "a"
        },
        {
            "amount": 53,
            "receiver": "b",
            "sender": "a"
        },
        {
            "amount": 1,
            "receiver": "Richard",
            "sender": "609febafd5104eafa33aecb4de9ea7b6"
        }
    ]
}
```

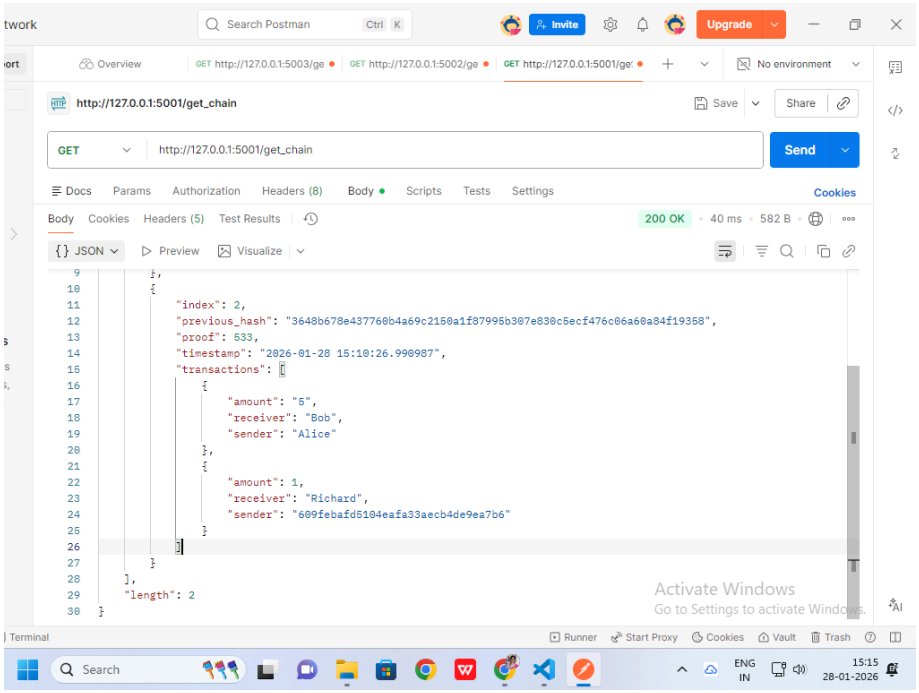


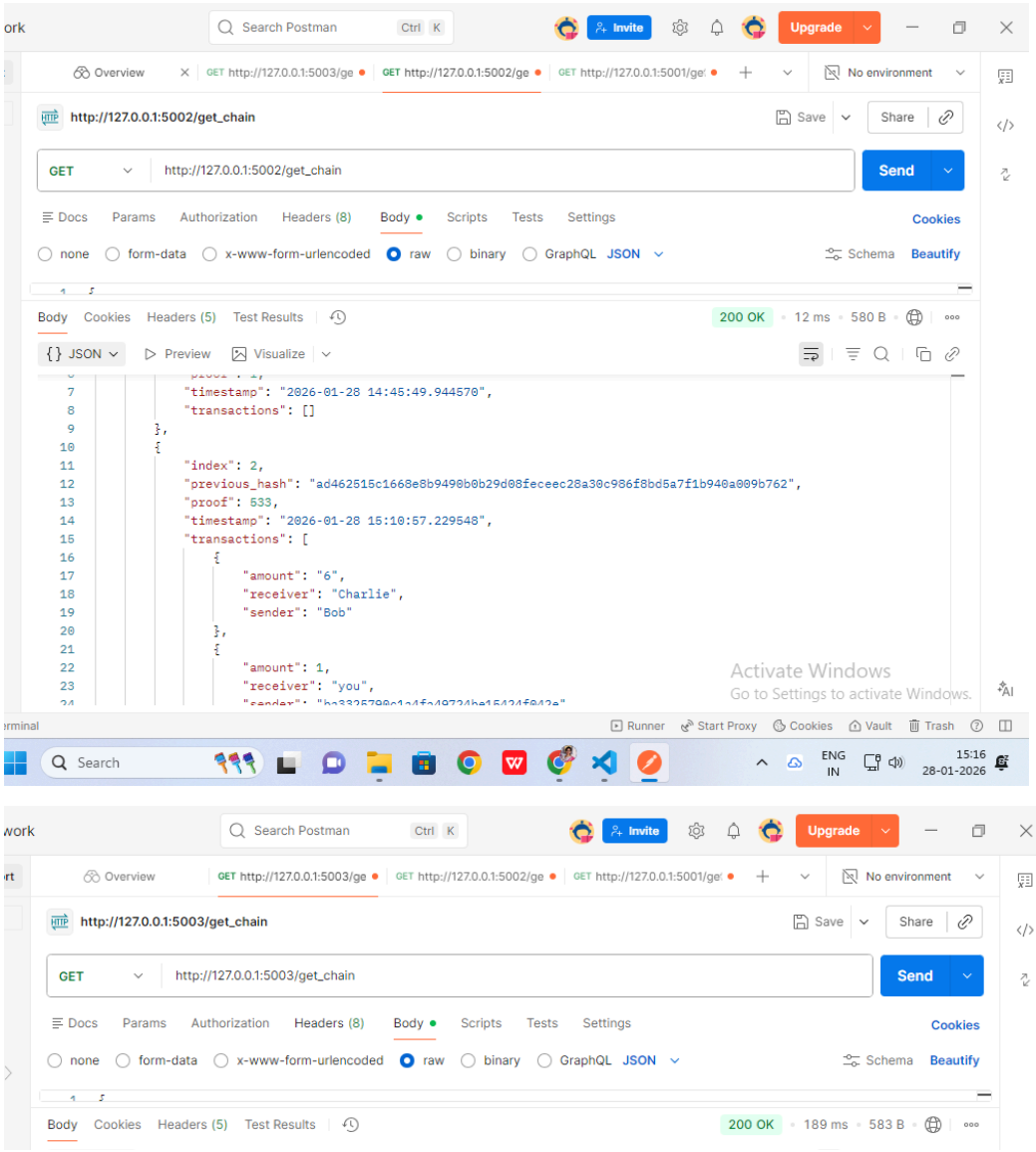


9. Mine blocks – GET /mine_block.



10. Fetch blockchain – GET /get_chain.





11. Replace chain – GET /replace_chain.

Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.